

Technical Requirements Document

1. Purpose

This document defines the technical architecture and implementation requirements for the Sanatan Dharma platform.

The platform must support:

- Bilingual content
- Scripture browsing
- Structured learning
- Search
- AI-assisted answers
- User accounts
- Bookmarks and learning progress
- Content verification
- Mobile-responsive design
- Future mobile application support

The architecture should be scalable without being unnecessarily complex for Version 1.

2. Recommended Technology Stack

Frontend

Recommended:

Next.js + React + TypeScript

Reasons:

- Strong SEO
- Excellent for content-heavy websites
- Supports server-side rendering
- Good performance
- Large ecosystem
- Suitable for future scaling
- Easy deployment
- Works well with AI applications

Styling

Recommended:

Tailwind CSS

Use with reusable design-system components.

Optional component library:

- shadcn/ui

The UI should still be customized heavily so the platform does not look generic.

3. Backend

Recommended approach:

Use Next.js server functionality initially.

This includes:

- Server Actions
- Route Handlers
- Server Components where appropriate

A completely separate backend is not required for the first version.

Future versions can split the backend into independent services if traffic or complexity requires it.

4. Database

Recommended:

PostgreSQL

Reasons:

The platform has highly relational data.

Examples:

- Scripture
- Chapter
- Verse
- Translation
- Commentary
- Deity
- Festival
- Mantra
- Source
- Tradition
- User
- Bookmark

PostgreSQL is well suited to these relationships.

5. Recommended Database Provider

For an MVP:

Supabase PostgreSQL

Advantages:

- Managed PostgreSQL
- Authentication available
- File storage
- APIs
- Row Level Security
- Good developer experience
- Free/low-cost starting tier

The architecture should avoid excessive Supabase-specific coupling where possible.

6. ORM

Recommended:

Drizzle ORM

Alternative:

Prisma.

Recommended initial choice:

Drizzle.

Reasons:

- Strong TypeScript support
 - Lightweight
 - Good SQL visibility
 - Suitable for PostgreSQL
-

7. Authentication

Recommended initial provider:

Supabase Auth

V1 authentication methods:

- Email
- Google

Possible later:

- Apple
- Phone number

Guest users should be able to access public educational content without logging in.

8. User Roles

Initial roles:

Visitor

Can:

- Read public content
- Search
- Use limited AI features

Registered User

Can:

- Bookmark
- Save verses
- Save mantras
- Track progress
- Manage preferences
- Use AI history

Editor

Can:

- Create content
- Edit drafts
- Add sources

Reviewer

Can:

- Review content
- Approve verification

Administrator

Can:

- Manage users
- Manage content
- Manage sources
- Publish content
- Manage platform settings

Role permissions should be enforced on the server.

9. Core Architecture

Recommended architecture:

User

↓

```
Next.js Web Application
↓
Server Layer
↓
PostgreSQL
↓
Search / AI / Storage Services
```

More detailed:

```
Browser
↓ HTTPS
Next.js
├── UI
├── Server Components
├── API Routes
├── Authentication
├── Business Logic
└── AI Layer
↓
PostgreSQL
├── Content
├── Users
├── Sources
├── Bookmarks
├── Learning Progress
└── AI Metadata
```

10. Content Architecture

The content system should store structured content instead of large uncontrolled blobs wherever possible.

Example relationship:

```
Scripture
↓
```

Chapter
↓
Verse
↓
Translation
↓
Commentary

Each element should have independent IDs.

11. Main Content Entities

The database should support:

- users
- profiles
- scriptures
- scripture_sections
- chapters
- verses
- translations
- commentaries
- commentators
- traditions
- concepts
- deities
- mantras
- festivals
- articles
- learning_paths
- learning_modules
- sources
- citations
- tags
- content_tags
- bookmarks
- reading_history
- user_progress
- ai_conversations
- ai_messages
- content_reviews

The exact schema will be defined in the next database document.

12. Internationalization

V1 languages:

- English
- Hindi

Recommended framework:

Next.js internationalization with a library such as:

next-intl

Content language should be stored separately from interface language.

Example:

```
locale = en  
locale = hi
```

The architecture should allow additional languages later.

13. URL Architecture

Recommended structure:

English:

```
/en/learn/karma  
/en/scriptures/bhagavad-gita/2/47
```

Hindi:

```
/hi/learn/karma  
/hi/scriptures/bhagavad-gita/2/47
```

Alternative:

English can be the default without `/en`.

For simplicity and future expansion, explicit locale URLs are recommended.

14. SEO Requirements

SEO is important because many users will discover the platform through search engines.

Each public content page must include:

- Unique page title
- Meta description
- Canonical URL
- Structured heading hierarchy
- Open Graph metadata
- Sitemap
- Robots configuration
- Structured data where appropriate

Important pages should render meaningful HTML without depending entirely on client-side JavaScript.

15. Structured Data

Where appropriate, add Schema.org structured data.

Possible types:

- Article
- BreadcrumbList
- FAQPage
- WebSite

Religious scriptures may require custom semantic handling rather than forcing inappropriate schema types.

16. Search System

Search must support:

- English
- Hindi
- Sanskrit
- Transliteration
- Alternative spellings

Search targets:

- Concepts
- Scriptures
- Chapters
- Verses
- Deities
- Mantras
- Festivals

17. V1 Search Recommendation

Start with:

PostgreSQL full-text search

Plus normalized searchable fields.

This avoids adding an unnecessary external search engine initially.

18. Future Search Upgrade

If search becomes more complex, consider:

- Typesense
- Meilisearch
- Algolia
- Elasticsearch/OpenSearch

Upgrade only when required by scale or advanced search needs.

19. Search Normalization

Create searchable forms for terms.

Example:

Krishna
Kṛṣṇa
कृष्ण

All should connect to the same entity.

Aliases should also be stored.

Example:

Shiva
Śiva
शिव
Mahadeva
Mahadev

20. AI Architecture

The AI system should not be a simple chatbot connected directly to a general model.

Recommended:

Retrieval-Augmented Generation (RAG)

Architecture:

```
Question
↓
Question processing
↓
Retrieve trusted content
↓
Rank sources
↓
Build context
↓
AI model
↓
Generate response
↓
Attach citations
```

↓

Return answer

21. AI Provider Abstraction

Do not tightly couple the entire product to one AI model provider.

Create an internal AI service layer.

Example:

```
AIService.generateAnswer()  
AIService.generateEmbedding()  
AIService.summarize()
```

This makes it easier to switch providers later.

22. AI Model Requirements

The selected model should perform well at:

- English
- Hindi
- Sanskrit terminology
- Long-context comprehension
- Citation-following
- Structured responses
- Retrieval-grounded generation

Exact provider and model should be selected closer to implementation because available models and pricing change frequently.

23. AI System Prompt Requirements

The system prompt must instruct the model to:

- Prefer retrieved sources
- Never fabricate scripture
- Clearly distinguish interpretation

- Cite sources
 - Admit uncertainty
 - Mention differences between traditions
 - Avoid pretending to be a spiritual authority
 - Avoid unsupported claims
-

24. AI Retrieval Corpus

The AI should retrieve only from approved content categories.

Possible corpus:

- Verified scripture
- Approved translations
- Approved commentaries
- Verified concept pages
- Verified festival content
- Verified mantra descriptions

Draft content should not enter production AI retrieval.

25. Vector Search

RAG will likely require embeddings.

Recommended future architecture:

pgvector inside PostgreSQL

Advantages:

- Keeps vectors near primary data
 - Simpler infrastructure
 - Appropriate for MVP
 - Works well with Supabase
-

26. Chunking Strategy

Long content should be broken into meaningful chunks.

Avoid arbitrary splitting.

Examples:

Scripture:

Verse-level or small verse groups.

Commentary:

Commentary tied to a verse.

Concept article:

Section-level chunks.

Festival:

Section-level chunks.

Every chunk should retain metadata.

Example:

```
source_id
content_id
content_type
scripture
chapter
verse
tradition
language
verification_status
```

27. AI Citation Pipeline

Each retrieved chunk should include a source ID.

The generated response should map claims back to those sources.

Users should be able to click:

Bhagavad Gita 2.47

and open the relevant page.

28. AI Conversation Storage

Logged-in users may save chat history.

Tables may include:

- conversations
- messages
- cited_sources

Users should be able to delete AI conversation history.

29. AI Usage Limits

Since AI costs money, V1 should support configurable usage limits.

Example:

Anonymous:

Limited questions per day.

Registered users:

Higher limit.

Future supporters:

Potentially higher limit.

These should be adjustable without changing application code.

30. AI Abuse Protection

Implement:

- Rate limiting
- Prompt length limits

- Request validation
 - Authentication where necessary
 - IP-based controls for anonymous usage
 - Abuse monitoring
-

31. Admin Content Management System

The project needs an internal CMS.

Two options:

Option A

Build a custom admin interface.

Option B

Use a headless CMS.

Recommended for V1:

Custom lightweight admin interface connected directly to the database.

Reasons:

The scripture/source structure is specialized and may not fit generic CMS systems naturally.

32. Admin Routes

Example:

```
/admin  
  
/admin/scriptures  
/admin/verses  
/admin/translations  
/admin/commentaries  
/admin/concepts  
/admin/deities  
/admin/mantras  
/admin/festivals
```



```
/admin/sources  
/admin/reviews  
/admin/users
```

Admin routes must require proper authorization.

33. Content Editor

Editors need:

- Rich text
- Markdown support
- Sanskrit fields
- Hindi fields
- English fields
- Source selector
- Citation insertion
- Tags
- Preview
- Draft saving
- Review workflow

34. Rich Text Format

Recommended:

Store structured rich text as JSON or Markdown depending on content type.

Do not store page-layout HTML directly unless necessary.

Scripture fields should remain structured database fields.

35. File Storage

Files may include:

- Images
- Audio
- PDFs
- Source scans

- Profile pictures

Recommended:

Supabase Storage initially.

Production source documents may require restricted access depending on copyright.

36. Image Optimization

All public images should use:

- Responsive sizing
- Modern image formats
- Lazy loading
- Compression
- Meaningful alt text

Next.js image optimization can assist.

37. Audio

Future mantra/scripture audio should support:

- Streaming
- Playback position
- Duration metadata
- Multiple reciters where appropriate
- Language/pronunciation metadata

Do not add heavy audio infrastructure to the first build unless required.

38. API Architecture

Create clean server endpoints for reusable operations.

Potential routes:

```
/api/search  
/api/ai/chat  
/api/bookmarks
```

```
/api/progress  
/api/content  
/api/sources
```

Internal server actions may replace some API endpoints for web-only interactions.

Public APIs are not required in V1.

39. Future Mobile API

Because mobile apps are planned later, business logic should not exist only inside browser components.

Critical actions should be reusable through server services/API boundaries.

This will make future mobile development easier.

40. Validation

Recommended:

Zod

Use for:

- API payloads
- Form submissions
- Environment variables
- Admin content validation

Never trust client-provided data directly.

41. Security

Minimum requirements:

- HTTPS
- Secure authentication
- Server-side authorization
- Input validation
- SQL injection protection

- XSS protection
 - CSRF protections where relevant
 - Rate limiting
 - Secure cookies
 - Environment variables
 - No secret keys sent to browsers
-

42. Row Level Security

If Supabase is used, Row Level Security should be enabled for user-specific tables.

Example:

A user can read their own bookmarks.

They cannot access another user's bookmarks.

Administrative operations should use secure server-side privileges.

43. Privacy

Collect only necessary personal information.

Initial profile may contain:

- Name
- Email
- Preferred language
- Avatar

Avoid unnecessary demographic or religious profiling.

44. Analytics

Recommended privacy-conscious analytics.

Possible V1 options:

- Vercel Analytics
- PostHog

- Plausible

Metrics:

- Page views
- Search activity
- Learning path engagement
- AI usage
- Bookmark activity
- User retention
- Performance

Avoid collecting sensitive religious-interest profiles unnecessarily.

45. Error Monitoring

Recommended:

Sentry

Track:

- Frontend errors
- Backend errors
- API failures
- AI failures

Do not send sensitive content unnecessarily to monitoring services.

46. Logging

Server logs should include:

- Request type
- Error information
- AI failures
- Admin actions
- Content publication activity

Never log:

- Passwords
- Authentication tokens
- Private keys

Sensitive chat content should be minimized in logs.

47. Hosting

Recommended V1 hosting:

Vercel

Suitable for Next.js.

Database:

Supabase.

Storage:

Supabase.

Possible architecture:

```
Vercel
├─ Next.js Application
└─ Server Functions

Supabase
├─ PostgreSQL
├─ Authentication
├─ Storage
└─ pgvector
```

48. Domain

A custom domain should be purchased before production launch.

Requirements:

- HTTPS
- Easy spelling
- Relevant brand identity
- Avoid confusing names
- Prefer `.com`, `.org`, or suitable Indian domain where appropriate

Final domain depends on chosen product name.

49. Environment Structure

Recommended:

```
Development
Staging
Production
```

Development:

Local coding.

Staging:

Testing before release.

Production:

Live public platform.

Never test major changes directly in production.

50. Source Control

Use:

Git

Recommended hosting:

GitHub

Branch strategy:

```
main
develop
```

```
feature/*  
fix/*
```

For a solo project, keep branching simple.

51. Continuous Deployment

Recommended:

GitHub → Vercel

Flow:

```
Code  
↓  
Git Commit  
↓  
GitHub  
↓  
Automated Checks  
↓  
Preview Deployment  
↓  
Review  
↓  
Production
```

52. Automated Quality Checks

Before deployment:

- TypeScript check
- ESLint
- Build test
- Unit tests
- Basic security checks

Later:

- End-to-end tests
- Accessibility checks

- Visual regression tests
-

53. Testing Frameworks

Recommended:

Unit/component testing:

Vitest

End-to-end testing:

Playwright

These should be introduced gradually.

54. Critical V1 Tests

At minimum test:

- Login
 - Logout
 - Search
 - Language switching
 - Verse loading
 - Previous/next verse
 - Bookmarking
 - Learning progress
 - AI question submission
 - AI citation display
 - Unauthorized admin access
-

55. Performance Targets

Target:

- Fast first load
- Responsive interactions
- Optimized images
- Minimal unnecessary client JavaScript

Aim for strong Core Web Vitals.

Public scripture pages should remain fast even without login.

56. Caching

Use caching for content that changes infrequently.

Examples:

- Scripture
- Concepts
- Deity pages
- Festival explanations

Avoid excessive caching for:

- User bookmarks
 - AI conversations
 - Personalized progress
-

57. Database Backups

Production database must have automated backups.

Content such as verified translations and source metadata is valuable and should not rely on one live database copy.

58. Migration Strategy

All database changes should use migrations.

Never manually change production database structure without recording the migration.

59. Accessibility

Technical implementation should target:

WCAG 2.1 AA

Requirements include:

- Semantic HTML
 - Keyboard support
 - ARIA only where necessary
 - Proper labels
 - Focus states
 - Contrast compliance
-

60. Browser Support

Support current versions of:

- Chrome
- Safari
- Edge
- Firefox

Mobile:

- Chrome Android
- Safari iOS

Do not optimize for obsolete browsers unless analytics later justify it.

61. Progressive Web App

PWA support can be added after the core website works.

Potential future functionality:

- Install to home screen
- Offline reading
- Saved scripture
- Notifications

Do not prioritize PWA over core content quality.

62. Future Native Apps

Possible future stack:

React Native / Expo

This would allow reuse of TypeScript knowledge and some business logic.

Do not build native apps during the initial web MVP.

63. Notifications

Future notifications may include:

- Daily shloka
- Learning reminder
- Festival reminder

Users must explicitly opt in.

Notifications should never become spammy.

64. Email

Potential transactional emails:

- Account verification
- Password reset
- Important account alerts

Future:

- Optional educational newsletter

Recommended providers can be selected later.

65. Feature Flags

Useful for releasing experimental features safely.

Examples:

AI assistant
Dark mode
Advanced commentary
Audio
Progress tracking

Features can be enabled gradually.

66. Content Versioning

Published content should store:

- Created date
- Updated date
- Reviewer
- Revision history
- Verification status

This supports transparency and corrections.

67. Admin Audit Log

Important admin operations should be logged.

Examples:

- Content published
 - Content removed
 - Source changed
 - Reviewer approval
 - User role changed
-

68. Religious Data Integrity

Scripture IDs, chapter numbers, verse numbers, and source references should have database constraints.

The system should make accidental duplicate or invalid scripture records difficult.

69. Data Separation

Keep:

Original text

separate from:

Translation

separate from:

Commentary

separate from:

AI explanation

This must be reflected in both database design and frontend display.

70. MVP Architecture Recommendation

Recommended complete MVP stack:

```
Frontend
Next.js
React
TypeScript
Tailwind CSS
shadcn/ui

Backend
Next.js Server Layer

Database
PostgreSQL

Database Hosting
Supabase

Authentication
Supabase Auth
```

ORM

Drizzle

Validation

Zod

AI

Provider abstraction + RAG

Vector Database

PostgreSQL + pgvector

Search

PostgreSQL full-text search

File Storage

Supabase Storage

Hosting

Vercel

Source Control

Git + GitHub

Testing

Vitest + Playwright

Monitoring

Sentry

Analytics

Vercel Analytics/PostHog/Plausible

71. What Should NOT Be Built Initially

Avoid premature complexity such as:

- Kubernetes
- Microservices
- Dedicated Elasticsearch cluster
- Separate backend application
- Multiple databases
- Custom authentication system

- Blockchain
- Complex recommendation algorithms
- Real-time community chat
- Native apps
- Large streaming infrastructure

The initial product does not require them.

72. Development Principle

The architecture should be:

Simple enough to build now, structured enough to grow later.

Avoid both:

Underengineering

and

building for millions of users before the first real user exists.

73. Implementation Phases

Phase 1 — Foundation

- Next.js project
- TypeScript
- Tailwind
- Design system
- Database
- Authentication

Phase 2 — Content

- Concepts
- Scriptures
- Gita
- Deities
- Mantras
- Festivals

Phase 3 — User Features

- Login
- Bookmarks
- Library
- Progress

Phase 4 — Search

- Search indexing
- Multilingual search
- Filters

Phase 5 — AI

- RAG
- Citations
- AI interface
- Rate limits

Phase 6 — Admin

- Content editor
- Source management
- Review workflow

Phase 7 — Quality

- Testing
- Performance
- Accessibility
- Security

Phase 8 — Launch

- Domain
 - SEO
 - Analytics
 - Monitoring
 - Production deployment
-

74. Open Technical Decisions

Still to be finalized:

- Exact AI provider
- Exact embedding provider
- AI usage limits
- Exact source licenses
- Search ranking algorithm
- Production email provider
- Audio infrastructure
- Analytics provider
- Domain
- Final deployment budget

These can remain open until implementation planning.

75. Status

Technical Requirements Document:

Initial Version Complete

The recommended architecture is now clear enough to move to the next document:

Database Schema & Data Model